



UNIVERSIDAD NACIONAL DE RÍO CUARTO

SISTEMAS OPERATIVOS

(Código 1965)

## **Taller 6: Procesos de usuario en EDOS**

Departamento de Computación

Facultad de Ciencias Exactas, Físico-Químicas y Naturales

**Alumno**

Tomás Rodeghiero

**Fecha de entrega:** 24 de junio de 2026

## Índice

|                                     |          |
|-------------------------------------|----------|
| <b>1. Objetivo</b>                  | <b>2</b> |
| <b>2. Estructura de la entrega</b>  | <b>2</b> |
| <b>3. Resolución por ejercicios</b> | <b>2</b> |
| <b>4. Conclusión</b>                | <b>8</b> |

## 1. Objetivo

Resolver el paso 08-process del proyecto EDOS. El paso agrega procesos de usuario: `create_process()` en `task.c` crea una task, le arma un page table propio con `exec()` que carga el binario del programa desde un filesystem embebido (EFS) y mapea código, datos y un user-stack. El primer proceso (`init`) arranca cuando el scheduler lo elige y `return_to_user_mode()` hace `sret` hacia U-mode. Los syscalls llegan vía `ecall`, disparan `u_trap` (`trap.s`), pasan por `user_trap()` en `trap.c` y son despachados por `syscall.c`.

El README.md pide nueve ejercicios: seis de análisis del código (1 a 6 y 8) y dos de modificación (7: provocar un page fault desde el init; 9: agregar un syscall `time` que devuelva los ticks transcurridos desde boot).

## 2. Estructura de la entrega

- 08-process/: paso original sin modificar, queda como referencia.
- entrega/respuestas.txt: respuestas a los nueve ejercicios del README.md.
- entrega/init-ejercicio7.c: `init.c` modificado para el ejercicio 7 (provoca un page fault escribiendo en `0xdeadbeef`).
- entrega/init-ejercicio9.c: `init.c` modificado para el ejercicio 9 (usa el nuevo syscall `time()`).
- entrega/syscall.c, entrega/syscall.h, entrega/edoslib.h, entrega/usys.s: archivos del kernel y de la edoslib modificados para el ejercicio 9.
- entrega/salida-original.txt, salida-ejercicio7.txt, salida-ejercicio9.txt: capturas de texto de QEMU.
- entrega/salida-ejercicio7.png, salida-ejercicio9.png: capturas de pantalla de la terminal con la corrida de cada ejercicio.
- informe-taller6-process.pdf: este informe.
- README.md: copia del enunciado del paso.

## 3. Resolución por ejercicios

### Ejercicio 1: bootear EDOS y ver el init

Compilé y corrí el paso 08-process. La salida es la de `entrega/salida-original.txt`:

```
Initializing kernel block allocator...
free pages=32759, free_list=80009000, kend=80009000, mem_end=88000000
ktext_start: 80000000
kdata_start: 80003000
mapping user code va=00000000 to pa=8002e000
Running scheduler on CPU 0
EDOS init process: pid=1!
Init going to sleep for 4 ticks...
pid 1 going to sleep.
Task pid=1 awake.
init awake! Finishing...
Freeing task init resources...
```

Leyendo el log de arriba para abajo: `init_kalloc()` arma la free list (32759 páginas libres con 128 MiB de RAM), `init_vm()` configura el page table del kernel, `create_process("init")` aloca y mapea código en `va=0` → `pa=0x8002e000` y deja la task `RUNNABLE`. El scheduler la elige, `start_process()` hace `return_to_user_mode()` y la primera instrucción del `init` corre en U-mode. El `init` imprime su `pid` (`=1`), entra a `sleep(4)`, el contador de ticks lo despierta cuatro ticks después, vuelve, termina con `exit()` y el scheduler libera recursos.

## Ejercicio 2: allocator de páginas físicas

Vive en `kalloc.c`. La estructura central es una free list simplemente enlazada cuyos nodos viven dentro de las páginas libres:

```
1 struct link { struct link *next; };
2 static struct link *free_list = NULL;
3 static unsigned int free_pages = 0;
```

`init_kalloc()` recorre la RAM desde `__mem_end - PAGE_SIZE` hacia abajo en pasos de `PAGE_SIZE` hasta `__kernel_end` y llama `free_page(p)` en cada página. Eso arma la lista y fija `free_pages` con la cantidad total. `alloc_page()` saca la cabeza de la lista, hace `memset(0)` sobre la página (limpia datos del kernel anteriores) y la devuelve. `free_page(pa)` valida alineación de página y rango y encadena al frente.

Es un allocator simple:  $O(1)$  en `alloc` y `free`, sin fragmentación externa (todas las páginas son del mismo tamaño), sin metadata aparte del puntero embebido en la página libre. En este paso no está protegido por un spinlock; en pasos siguientes habría que serializarlo.

## Ejercicio 3: layout del espacio de direcciones del kernel

`map_kernel_memory()` en `arch.c` configura mapeos 1:1 (`va = pa`) de todo lo que necesita ver el kernel con paginación encendida:

```
1 map_region(pgtbl, UART, UART, PAGE_SIZE, R|W);
2 map_region(pgtbl, VIRTIO0, VIRTIO0, PAGE_SIZE, R|W);
3 map_region(pgtbl, PLIC, PLIC, PLIC_SIZE, R|W);
4 map_region(pgtbl, ktext_start, ktext_start, ktext_size, R|X);
5 map_region(pgtbl, kdata_start, kdata_start, kdata_size, R|W);
```

Diagrama del layout resultante:

```
Direcciones logicas del kernel (1:1 con fisicas)

+-----+ 0x00000000
| no mapeado |
+-----+ 0x0c000000
| PLIC (R/W) | 4 MiB (PLIC_SIZE)
+-----+ 0x0c400000
| no mapeado |
+-----+ 0x10000000
| UART (R/W) | 1 pagina
+-----+ 0x10001000
| VIRTIO0 (R/W) | 1 pagina
+-----+ 0x10002000
| no mapeado |
+-----+ 0x80000000 __kernel_start
| kernel .text (R/X) |
+-----+ 0x80003000 __text_end (en mi corrida)
| .rodata, .data, .bss, |
| stacks por CPU y resto |
```

```
| de la RAM (R/W) |
+-----+ 0x88000000 __mem_end (128 MiB)
```

Los MMIO (UART, VIRTIO, PLIC) van como R|W, nunca con PAGE\_X. El `.text` del kernel queda R|X y el resto de la RAM R|W: el kernel no puede ejecutar datos por error. Como los mapeos son 1:1, `enable_paging()` se limita a cargar `satp` con el page table; los símbolos no cambian de dirección. Cada page table de proceso copia este page table del kernel como base y le agrega los mapeos U-mode en `[0, PROC_MAX_VA)`.

#### Ejercicio 4: pasos de `create__process()`

`create_process(path)` en `task.c`, en orden:

1. **`create_task(path, start_process)`**: busca un slot UNUSED en `tasks[]` (con lock), zera el contexto y deja `ctx.ra = start_process`. Aloca `kstack` con `alloc_page()` y deja `ctx.sp = kstack + PAGE_SIZE`. Estado CREATED.
2. Asigna `t->pid = ++last_pid` (al init le toca 1, que es lo que imprime `getpid` en el ej. 1).
3. **`exec(t, path, 0)`**:
  - Aloca `new_pgtbl` y le copia encima el `kernel_pgtbl` (`memcpy`).
  - `load_program(new_pgtbl, path)`: con `efs_file(path)` obtiene el binario, aloca una página por cada `PAGE_SIZE` de código, copia los bytes y mapea `va → pa` con R|W|X|U. El log "mapping user code va=00000000 to pa=..." viene de acá.
  - Aloca el user-stack y lo mapea en `PROC_MAX_VA - PAGE_SIZE` con R|W|U.
  - `setup_main_args()`: pone `argc` y `argv` en `a0/a1` del trap frame.
  - Inicializa el trap frame: `tf->sp = sp` (top del user-stack), `tf->pc = 0` (entry point del programa).
  - Si la task ya era proceso, libera el page table anterior.
  - `task->pgtbl = new_pgtbl`.
  - `task->ctx.sp = (size_t) tf`: al hacer el primer `context_switch`, el `sp` del task va a apuntar al trap frame en la `kstack`.
4. `t->state = RUNNABLE`. El scheduler ya puede elegir la tarea.

#### Ejercicio 5: cómo se compilan los programas de usuario

(a) Linker script `user.ld`:

```
1 ENTRY(start)
2
3 SECTIONS {
4     . = 0;
5     .text : { KEEP(*(.text.start)); *(.text .text.*); }
6     .rodata : ALIGN(4) { *(.rodata .rodata.*); }
7     .data : ALIGN(4) { *(.data .data.*); }
8     .bss : ALIGN(4) { *(.bss .bss.* .sbss .sbss.*); }
9 }
```

El linker arranca la imagen en `va=0` (es lo que espera `exec()` cuando hace `tf->pc = 0`). `KEEP` de `.text.start` asegura que esa sección no sea descartada; combinado con el orden de lindeo

del Makefile del user (edolib.o usys.o init.c), la primera función ubicada en `.text` es `start()` de `edolib.c`.

**(b) Entry point.** `ENTRY(start)` marca `start` como punto de entrada. El kernel ignora la cabecera ELF porque el Makefile hace `objcopy -O binary init init.bin` y sólo eso termina embebido en `efsfiles.c`. Lo único que importa es que `start()` caiga en `va=0`. La función está en `edolib.c`:

```
1 void start(void) {
2     exit(main());
3 }
```

Cuando `exec()` arma `tf->pc = 0` y `sret` devuelve el control a U-mode, la CPU ejecuta `start()`, `start` llama al `main()` del programa y, cuando `main` retorna, `exit()` (syscall 0) le pasa el código de salida al kernel.

**(c) mkefs.sh.** El script (lo invoca el Makefile del user con `./mkefs.sh init README`) arma `efsfiles.c`:

1. Crea un temporal con `#include "efs.h"`.
2. Para cada nombre, si existe `<nombre>.bin` (caso del programa `init.bin`) usa `xxd -i <nombre>.bin`; si no (caso del archivo `README`) usa `xxd -i <nombre>.xxd -i` genera el arreglo de bytes y la longitud.
3. Reemplaza `unsigned int` por `const unsigned int` en las longitudes (sed) para que el linker las ubique en `.rodata`.
4. Abre la tabla `efs_files_table[]` y emite una fila por archivo: `{"nombre", EFS_FILE_PROGRAM, ...}` o `EFS_FILE_DATA`.
5. Cierra con un centinela `{0, 0, 0, 0}` y mueve `efsfiles.c` al directorio del kernel.

En tiempo de compilación del kernel, `efs_file(name)` (`efs.c`) recorre esa tabla por `strcmp` y devuelve un puntero al binario y su longitud. El filesystem queda en RAM, embebido en el `.rodata` del kernel.

## Ejercicio 6: del scheduler a la primera instrucción del usuario

Recorrido completo de la primera vez que se elige al init:

1. `scheduler()` encuentra el slot `RUNNABLE`, lo pasa a `RUNNING` y llama `context_switch(&cpu_state[cpu].ctx, &next_task->ctx)`.
2. `context_switch (arch.s)` restaura del contexto del task: `sp = kstack + PAGE_SIZE` y `ra = start_process.ret` salta a `start_process`.
3. `start_process()` libera el lock que tenía el scheduler y llama `return_to_user_mode()`.
4. `return_to_user_mode()` (`trap.c`):

```
1 disable_interrupts();
2 set_u_trap_handler(); // stvec <- u_trap
3 set_kstack(kstack+PAGE_SIZE); // sscratch <- top kstack
4 set_u_previous_mode(); // sstatus.SPP=0, SPIE=1
5 set_page_table(task->pgtbl); // satp <- pa2satp(pgtbl)
6 u_trap_ret(task_trap_frame_address(task));
```

5. `u_trap_ret (trap.s)`: pone `sp` en el trap frame, restaura `sepc` desde `tf->pc (= 0)`, restaura los 29 registros, carga `sp = tf->sp` (top del user-stack) y hace `sret`.

6. `sret` pasa a U-mode (`SPP=0`), habilita interrupciones (`SPIE=1`) y carga `pc ← sepc = 0`. Como el `pgtbl` del proceso mapea `va=0` al código del `init`, la primera instrucción ejecutada en U-mode es la primera del `.text` del `init.bin`, que es la primera instrucción de `start()`.

## Ejercicio 7: provocar un page fault desde el init

Modifiqué `init.c` (entrega/init-ejercicio7.c) para que el proceso intente escribir en una dirección no mapeada de su espacio:

```
1 int main(void) {
2     printf("EDOS init process: pid=%d!\n", getpid());
3
4     printf("Intentando escribir en una direccion no mapeada "
5           "(0xdeadbeef)...\n");
6     volatile int *bad = (int *) 0xdeadbeef;
7     *bad = 0x42;
8
9     console_puts("Esta linea no deberia ejecutarse.\n");
10    return 0;
11 }
```

**Por qué falla.** El proceso sólo tiene mapeadas (con `PAGE_U`) las páginas de `[0, tamaño programa)` y la página del stack en `[PROC_MAX_VA - PAGE_SIZE, PROC_MAX_VA)`, con `PROC_MAX_VA = 0x80000000`. `0xdeadbeef` cae fuera de `PROC_MAX_VA`; aunque estuviera por debajo, no estaría mapeada. El store dispara una excepción `STORE_PAGE_FAULT` (`scause = 0x0F`). `user_trap()` cae en la rama de page fault y marca la task con `killed = true`.

**Verificación** (entrega/salida-ejercicio7.txt):

```
EDOS init process: pid=1!
Intentando escribir en una direccion no mapeada (0xdeadbeef)...
Task init in CPU 0 page fault. Killing task...
Freeing task init resources...
```

La línea "Esta linea no deberia ejecutarse" no aparece, como esperaba.

```
tomasrodeghiero@Tomass-MacBook-Pro-4 edos-build % qemu-system-riscv32 -machine virt -bios none -display none -serial file:"$SENTREGA/salida-ejercicio7.txt" -smp 1 -kernel kernel & QPID=$!;
sleep 6; kill -9 $QPID 2>/dev/null; wait $QPID 2>/dev/null; cat "$SENTREGA/salida-ejercicio7.txt"
[1] 8093
(f1) * Killed      qemu-system-riscv32 -machine virt -bios none -display none -serial -smp 1
Initializing kernel block allocator...
free pages=32759, free_list=80009000, kend=80009000, mem_end=88000000
ktext_start: 80000000
kdata_start: 80003000
mapping user code va=00000000 to pa=8002e000
Running scheduler on CPU 0
EDOS init process: pid=1!
Intentando escribir en una direccion no mapeada (0xdeadbeef)...
Task init in CPU 0 page fault. Killing task...
Freeing task init resources...
tomasrodeghiero@Tomass-MacBook-Pro-4 edos-build %
```

Captura de terminal: ejecución del ej. 7 en QEMU.

## Ejercicio 8: camino completo del syscall getpid()

Desde el llamador en U-mode hasta el retorno:

1. El código del usuario llama `getpid()`, declarado como `extern` en `edoslib.h`.
2. Stub en `user/usys.s`:

```
1 getpid:
2     li a7, 1 # SYS_GETPID
3     ecall
4     ret
```

Carga el número de syscall en `a7` y ejecuta `ecall`. `ecall` produce una excepción con `scause = 0x8` y `sepc = pc` del `ecall`.

3. `u_trap (trap.s)`: hace `csrrw sp, sscratch, sp` (cambia a la `kstack` del proceso), reserva espacio para el `trap_frame`, guarda `ra`, `gp`, `t0-t6`, `a0-a7`, `s0-s11`, el `sp` de usuario (leído de `sscratch`) y `sepc` en `tf->pc`. Después `call user_trap`.
4. `user_trap (trap.c)`: `set_page_table(kernel_pgtbl)`, `set_k_trap_handler()`, y para `case SYSCALL`: `enable_interrupts(); syscall(task); skip_trap_instruction(tf)` (`tf->pc += 4`, para que al volver no re-ejecute el `ecall`).
5. `syscall() (syscall.c)`: obtiene `n = tf->a7 = 1`, busca `syscalls_table[1]` (= `sys_getpid`), lo invoca y deja el resultado con `syscall_put_result(tf, result)` en `tf->a0`.
6. `user_trap` llama `return_to_user_mode()`, que reconfigura el entorno (`stvec`, `sscratch`, `sstatus`, `satp`) y `u_trap_ret` restaura los registros, pone `sepc <- tf->pc` (ya apunta a la instrucción siguiente al `ecall`) y hace `sret`.
7. En U-mode la CPU ejecuta la instrucción siguiente al `ecall`, que es el `ret` del stub `getpid`. Como el resultado quedó en `a0`, el llamador C recibe el valor de retorno normalmente.

## Ejercicio 9: agregar el syscall `time()`

Decidí implementar `time()` devolviendo el contador global `ticks` del kernel (`volatile unsigned int ticks` en `task.c`): es lo natural, ya lo incrementa `inc_ticks()` en cada timer interrupt y hay `get_ticks()` que lo lee bajo `ticks_lock`.

**Cambios** (todos en entrega/):

(1) `syscall.h`:

```
1 #define SYS_TIME 6
2 #define SYSCALLS 7
```

(2) `syscall.c`: nuevo handler y entrada en la tabla:

```
1 // time(): ticks transcurridos desde el boot
2 int sys_time(struct task *task)
3 {
4     (void) task;
5     return (int) get_ticks();
6 }
7
8 static syscall_f syscalls_table[SYSCALLS] = {
9     sys_exit, sys_getpid, sys_console_puts,
10    sys_console_putc, sys_console_getc, sys_sleep,
11    sys_time
12};
```

(3) `user/edolib.h`:

```
1 extern int time(void);
```

(4) `user/usys.s`:

```
1 .global time
2 time:
```



```

3  li a7, 6 # SYS_TIME
4  ecall
5  ret

```

#### (5) user/init.c:

```

1  int main(void) {
2      printf("EDOS init process: pid=%d!\n", getpid());
3      for (int i = 0; i < 5; i++) {
4          printf("init: time = %d ticks\n", time());
5          sleep(2);
6      }
7      printf("init: time final = %d ticks. Finishing...\n", time());
8      return 0;
9  }

```

Usar `get_ticks()` en lugar de leer `ticks` directamente evita una race con `inc_ticks()` cuando hay más de una CPU. La firma del handler devuelve `int` para mantener compatibilidad con el dispatcher (`typedef int (*syscall_f)(struct task *)`).

Verificación (entrega/salida-ejercicio9.txt):

```

EDOS init process: pid=1!
init: time = 0 ticks
pid 1 going to sleep.
Task pid=1 awake.
init: time = 2 ticks
pid 1 going to sleep.
Task pid=1 awake.
init: time = 4 ticks
...
init: time final = 10 ticks. Finishing...
Freeing task init resources...

```

El contador sube de a 2 entre cada print porque entre llamadas el proceso duerme 2 ticks. Eso confirma que el syscall está bien numerado, el stub pasa por la tabla y el resultado viaja en `a0` hasta el llamador en U-mode.

```

tomasrodeghiero@Tomass-MacBook-Pro-4 edos-build % qemu-system-riscv32 -machine virt -bios none -display none -serial file:"$ENTREGA/salida-ejercicio9.txt" -smp 1 -kernel kernel & QPID=$!;
sleep 12; kill -9 $QPID 2>/dev/null; wait $QPID 2>/dev/null; cat "$ENTREGA/salida-ejercicio9.txt"
[1] 8151
[1] + Killed          qemu-system-riscv32 -machine virt -bios none -display none -serial -smp 1
Initializing kernel block allocator...
free pages=32759, free_list=80009000, kend=80009000, mem_end=88000000
ktext_start: 80000000
kdata_start: 80003000
mapping user code va=00000000 to pa=8002e000
Running scheduler on CPU 0
EDOS init process: pid=1!
Intentando escribir en una direccion no mapeada (0xdeadbeef)...
Task init in CPU 0 page fault. Killing task...
Freeing task init resources...
tomasrodeghiero@Tomass-MacBook-Pro-4 edos-build %

```

Captura de terminal: ejecución del ej. 9 en QEMU.

## 4. Conclusión

El paso conecta varias piezas de los talleres anteriores (*spinlocks*, *kernel tasks* con preempción, *context switch*, `printf` compartido) con tres nuevas: allocator de páginas, page tables por proceso y traps de usuario. La parte conceptualmente más densa es el viaje `scheduler` → `start_process` → `return_to_user_mode` → `u_trap_ret` → `sret`, porque combina cosas que están en C, en assembler y en CSRs de la CPU al mismo tiempo.